

Numerical Validation of a Pure Python Implementation of k-NN Entropy Estimation Against a Reference C/ANN Library

Nicolas B. Garnier¹

Martin G. Frasch²

February 19, 2026

¹Laboratoire de Physique, CNRS, ENS de Lyon, Université de Lyon, Lyon, France

²Department of Obstetrics and Gynecology, University of Washington, Seattle, WA, USA

Abstract

We present a systematic validation of a pure Python implementation of k-nearest-neighbor (k-NN) based entropy and entropy-rate estimators against a mature reference implementation in C that uses the Approximate Nearest Neighbor (ANN) library. Testing on 251 segments of RR-interval time series from ambulatory ECG recordings, we identify and resolve three sources of numerical discrepancy: (i) the ANN library’s exclusion of all zero-distance points from neighbor searches, (ii) a mismatch between L_2 and L_∞ norms in high-dimensional k-d tree queries, and (iii) a misalignment of data points between the marginal-entropy and mutual-information terms in the entropy-rate computation at time scales $\tau > 1$. After correction, Shannon entropy estimates match to machine precision ($R^2 = 1.0000$) and entropy-rate estimates agree to within 0.06% ($R^2 = 0.9998$) across all tested time scales, confirming numerical equivalence of the two codebases. These findings highlight subtle implementation pitfalls in k-NN information-theoretic estimators that are relevant to any reimplementing effort.

1 Introduction

Entropy and entropy-rate estimation from finite samples is central to nonlinear time-series analysis in physiology, neuroscience, and engineering [Kozachenko and Leonenko, 1987, Kraskov et al., 2004]. The Kozachenko–Leonenko (KL) estimator [Kozachenko and Leonenko, 1987] and the Kraskov–Stögbauer–Grassberger (KSG) mutual information estimator [Kraskov et al., 2004] provide consistent, bias-corrected estimates based on k-nearest-neighbor (k-NN) distances. While conceptually straightforward, their numerical implementation involves several choices—distance metric, treatment of zero-distance neighbors, time-delay embedding alignment—that can introduce systematic offsets when different codebases are compared.

We describe the validation of a new pure Python implementation (`entropy_pure`) against an established C implementation that relies on the ANN library [Mount and Arya, 2010] for k-d tree operations. The reference C code has been used extensively in published studies of heart-rate variability (HRV) and fetal monitoring [Mayer et al., 2025].

The Python reimplementation aims to provide a dependency-free, readable, and extensible alternative suitable for research prototyping and teaching, while producing numerically identical results.

The validation uncovered three implementation-level discrepancies that collectively produced offsets of up to 14% in entropy-rate estimates. We document these discrepancies, their root causes, and the corrections applied, as a practical guide for developers of information-theoretic estimation libraries.

2 Methods

2.1 Kozachenko–Leonenko entropy estimator

Given N i.i.d. samples $\{x_1, \dots, x_N\}$ in $\mathbb{R}^d$, the KL estimator of the differential entropy $H(X)$ is

$$\hat{H}(X) = d \cdot \frac{1}{N_v} \sum_{\substack{i=1 \\ \varepsilon_i > 0}}^N \log(2\varepsilon_i) + \psi(N_v) - \psi(k), \quad (1)$$

where ε_i is the distance from x_i to its k -th nearest neighbor, $\psi(\cdot)$ is the digamma function, and $N_v \leq N$ is the number of points with $\varepsilon_i > 0$. The factor of 2 converts the distance to the diameter of the L_∞ ball, and the digamma terms provide the bias correction.

Two choices in this estimator have significant numerical impact:

1. **Distance metric.** The C implementation uses the ANN library compiled with L_∞ (Chebyshev) norm (`ANN_METRIC = LINF`). While $L_2 = L_\infty$ in one dimension, the two norms diverge for $d \geq 2$, producing a systematic offset in the Shannon entropy H at embedding dimension $m \geq 2$.
2. **Zero-distance handling.** When `ANN_ALLOW_SELF_MATCH = false`, the ANN k-d tree search excludes *all* points at distance zero from the query result—not merely the query point itself. For integer-valued data with many duplicate values (common in RR-interval time series quantized to 1 ms), this means the k -th neighbor always has $\varepsilon_i > 0$, whereas a standard k-d tree (e.g., `scipy.spatial.KDTree`) returns exact zeros for duplicate points.

2.2 KSG mutual information estimator

For the entropy-rate computation via method 2 ($h = H(X_t) - \text{MI}(X_t; X_{\text{past}})$), we use the KSG estimator (Algorithm 1):

$$\hat{I}(X; Y) = \psi(k) - \frac{1}{N} \sum_{i=1}^N [\psi(n_{x,i} + 1) + \psi(n_{y,i} + 1)] + \psi(N), \quad (2)$$

where $n_{x,i}$ and $n_{y,i}$ count the number of points within the L_∞ -ball of radius ε_i in each marginal space, with ε_i the L_∞ distance to the k -th neighbor in the joint space.

2.3 Entropy rate at multiple time scales

The entropy rate at time scale τ is

$$\hat{h}(\tau) = \hat{H}(X_t) - \hat{I}(X_t; X_{\text{past}}^{(m)}), \quad (3)$$

where $X_{\text{past}}^{(m)} = (X_{t-\tau}, X_{t-2\tau}, \dots, X_{t-m\tau})$ is the m -dimensional past embedding with lag τ .

The reference C implementation constructs a *unified embedding* of $m + 1$ dimensions via its `Theiler_embed` function: for each selected time index t , the embedded vector is $(x_t, x_{t-\tau}, \dots, x_{t-m\tau})$. Crucially, consecutive embedded vectors step by one time unit (the Theiler window $\Delta_T = 1$), not by τ . Both $\hat{H}(X_t)$ and $\hat{I}(X_t; X_{\text{past}})$ operate on the *same* set of N_{eff} points extracted from this unified embedding, ensuring algebraic consistency of the subtraction in Eq. (3).

2.4 Data and experimental setup

We use 251 five-minute segments of RR-interval time series extracted from ambulatory fetal ECG recordings. Segment lengths range from 139 to 547 beats (mean 422). The RR intervals are integer-valued in milliseconds, with substantial duplicate values: a typical 200-point subsample contains approximately 120 unique values, causing $\sim 40\%$ of k -th neighbor distances to be exactly zero when using a standard k-d tree.

All comparisons use $k = 5$, $N_{\text{eff}} = 200$, $N_{\text{real}} = 1$, embedding dimension $m = 2$, and explicit Theiler window $\Delta_T = 1$ (“type 0” in the C code’s sampling-parameter system). Time scales tested are $\tau \in \{1, 2, 4\}$.

3 Results

3.1 Discrepancy 1: Zero-distance duplicate exclusion

3.1.1 Observation

The one-dimensional marginal entropy $H(X_t)$ at $\tau = 1$ showed a mean offset of 0.43 nats between the Python and C implementations ($R^2 = 0.836$). Decomposing the entropy rate $\hat{h} = \hat{H}(X_t) - \hat{I}$ revealed that 99.6% of the entropy-rate offset originated in $\hat{H}$, not in $\hat{I}$.

3.1.2 Root cause

The ANN library’s k-d tree search (`kd_search.cpp`, line 234) contains the guard

```
if (d >= ANNkdDim && (ANN_ALLOW_SELF_MATCH || dist != 0))
    ANNkdPointMK->insert(dist, bkt[i]);
```

With `ANN_ALLOW_SELF_MATCH = false`, the condition `dist != 0` excludes *all* points at zero distance from the result set—not only the query point. For integer-valued data with many duplicates, this guarantees $\varepsilon_i > 0$ for every query, so $N_v = N_{\text{eff}}$ in Eq. (1).

In contrast, Python’s `scipy.spatial.KDTree` returns exact-zero distances for duplicate points. The original Python code excluded these as “errors,” reducing N_v and shifting the $\psi(N_v)$ term. With $k = 5$ and typical data: $\psi(123) - \psi(200) \approx -0.49$, closely matching the observed offset.

Algorithm 1 Zero-distance-aware k-NN entropy estimation

Require: Data $X \in \mathbb{R}^{d \times N}$, neighbors k

- 1: $d_{\max} \leftarrow \max(\text{duplicate counts via UNIQUE})$
 - 2: $k_{\text{ext}} \leftarrow \min(N, k + d_{\max})$
 - 3: $D \leftarrow \text{KDTree}(X).\text{QUERY}(X, k_{\text{ext}}, p = \infty)$ $\triangleright N \times k_{\text{ext}}$ distance matrix
 - 4: $n_{\text{zero},i} \leftarrow \sum_j \mathbb{1}[D_{ij} = 0]$ $\triangleright$ zeros per row (incl. self)
 - 5: $\varepsilon_i \leftarrow D_{i, n_{\text{zero},i} + k - 1}$ $\triangleright k$ -th non-zero distance
 - 6: $N_v \leftarrow \sum_i \mathbb{1}[\varepsilon_i > 0]$
 - 7: **return** $d \cdot \log(2\varepsilon)_{i:\varepsilon_i > 0} + \psi(N_v) - \psi(k)$
-

3.1.3 Correction

We modified `_entropy_knn` to count the maximum number of co-located points via `np.unique`, request $k + d_{\max}$ neighbors (where $d_{\max}$ is the maximum duplicate count), and for each query point select the k -th neighbor with non-zero distance. This is implemented via vectorized NumPy operations (Algorithm 1).

After correction, $H(X_t)$ matches to within numerical precision ($R^2 = 1.000$, mean absolute difference $< 10^{-12}$).

3.2 Discrepancy 2: L_2 vs. L_∞ norm

3.2.1 Observation

Shannon entropy H at embedding dimension $m = 2$ (a two-dimensional quantity) showed a consistent offset of 0.25 nats across all time scales, despite excellent correlation ($R^2 = 0.9996$).

3.2.2 Root cause

The ANN library is compiled with `ANN_METRIC = LINF` (ANN.h, lines 382–387), using the Chebyshev (L_∞) norm for all distance computations. Python’s `scipy.spatial.KDTree.query` defaults to L_2 (Euclidean). In one dimension $L_2 \equiv L_\infty$, so the marginal entropy $H(X_t)$ was unaffected. However, for $d = 2$ the norms differ: $\|x\|_\infty \leq \|x\|_2 \leq \sqrt{d} \|x\|_\infty$, producing systematically larger distance estimates under L_2 and thus a positive entropy bias.

3.2.3 Correction

We added `p=np.inf` to the `tree.query()` call in `_entropy_knn`, selecting the Chebyshev norm. After correction, Shannon entropy matches to machine precision across all scales ($R^2 = 1.0000$, Table 1).

3.3 Discrepancy 3: Embedding misalignment in entropy rate

3.3.1 Observation

After correcting Discrepancies 1 and 2, the entropy rate at $\tau = 1$ matched well ($R^2 = 0.9989$), but at $\tau = 2$ and $\tau = 4$ significant offsets persisted (mean absolute differences of 0.24 and 0.19, $R^2 \approx 0.85$; Table 2).

Table 1: Comparison of Python and C implementations after all corrections. Shannon entropy $H(m=2)$ and entropy rate $h(m=2)$ across 250 valid segments at three time scales.

Metric	Scale	Mean abs. diff.	Mean rel. %	Max abs. diff.	R^2
Shannon H	$\tau = 1$	$< 10^{-12}$	0.00	$< 10^{-12}$	1.0000
	$\tau = 2$	$< 10^{-12}$	0.00	$< 10^{-12}$	1.0000
	$\tau = 4$	$< 10^{-12}$	0.00	$< 10^{-12}$	1.0000
Entropy rate h	$\tau = 1$	0.0029	0.06	0.1084	0.9997
	$\tau = 2$	0.0014	0.03	0.0371	0.9999
	$\tau = 4$	0.0007	0.01	0.0242	0.9999

3.3.2 Root cause

The Python `compute_entropy_rate` (method 2) computed $\hat{H}(X_t)$ and $\hat{I}(X_t; X_{\text{past}})$ via separate function calls. Two problems emerged at $\tau > 1$:

1. **Different data for H and I .** The call `compute_entropy(x, n_embed=1, stride= $\tau$ )` selected the first N_{eff} consecutive points of the raw signal (since `pts_offset` = $\tau \cdot (1 - 1) = 0$ for `n_embed` = 1). Meanwhile, the MI computation used points from a time-delay embedding that starts at offset $\tau \cdot m$. The subtraction $\hat{h} = \hat{H} - \hat{I}$ is only meaningful when both terms operate on the same data points.
2. **Misaligned current/past pairing.** The past vectors were generated by `_embed_data(x, m,  $\tau$ )`, which steps by τ between consecutive embedded vectors. However, the “current” values `x_curr` = `x[:, offset:]` stepped by 1. At $\tau = 2$ this caused pairing errors: the i -th current value x_{4+i} was paired with the past vector for time $2i + 4$, matching only for $i = 0$.

The C implementation avoids both problems by constructing a single unified embedding of $(m + 1)$ dimensions via `Theiler_embed`, from which the current values (first row) and past values (remaining rows) are extracted. All N_{eff} embedded vectors step by $\Delta_T = 1$ (the Theiler window), with temporal lags of τ between embedding dimensions.

3.3.3 Correction

We rewrote `compute_entropy_rate` method 2 to construct the unified embedding directly:

1. Compute $t_i = \tau \cdot m + i$ for $i = 0, \dots, N_{\text{eff}} - 1$ (consecutive time indices).
2. Extract $X_{\text{curr}} = \{x_{t_i}\}$ and $X_{\text{past}} = \{(x_{t_i - \tau}, \dots, x_{t_i - m\tau})\}$.
3. Compute $\hat{H}$ on X_{curr} and $\hat{I}$ on $(X_{\text{curr}}, X_{\text{past}})$.

After correction, entropy-rate estimates agree to within 0.06% at all scales (Table 1).

Table 2: Progression of entropy-rate agreement as each discrepancy is corrected (values at $\tau = 1$).

Stage	Mean abs. diff.	R^2	Primary cause
Before any corrections	0.4302	0.836	All three
After zero-distance fix	0.0113	0.9989	Norm + alignment
After L_∞ norm fix	0.0029	0.9997	Alignment (minor at $\tau=1$)
After embedding alignment	0.0029	0.9997	Residual floating-point

3.4 Final validation results

Shannon entropy $H(m=2)$ now matches to machine precision at all tested time scales (Table 1). The residual entropy-rate difference of ~ 0.003 at $\tau = 1$ reflects minor differences in the marginal-count step of the KSG estimator (Algorithm 1), where the C code uses an iterative point-counting approach inside the ANN tree structure, while Python uses `scipy's query_ball_point`. At $\tau = 2$ and $\tau = 4$ the agreement is even tighter (mean difference < 0.002), likely because the coarser embedding reduces sensitivity to counting-boundary effects.

4 Discussion

4.1 Practical implications

The three discrepancies identified in this work are not bugs in the mathematical sense—both the C and the original Python code implement valid entropy estimators—but they prevent numerical reproducibility across codebases. We summarize the lessons for implementers:

1. **Self-match exclusion vs. duplicate exclusion.** The ANN library's `dist != 0` guard was designed to exclude the query point from its own neighbor list. However, it also excludes all co-located (duplicate) points. Any reimplementations that wish to match ANN's behavior must replicate this side effect explicitly. For continuous-valued data without ties, the distinction is immaterial; for quantized data (integer RR intervals, discrete event times), it dominates the entropy estimate.
2. **Distance metric consistency.** The KL estimator's $\log(2\varepsilon)$ term assumes the L_∞ ball volume $V_d = (2\varepsilon)^d$. When L_2 distances are used instead, the estimator implicitly uses L_2 -ball volumes $V_d = \pi^{d/2}\varepsilon^d/\Gamma(d/2 + 1)$, introducing a dimension-dependent bias. The L_∞ norm is both the correct choice for the KL estimator as commonly stated and the one used by the ANN library.
3. **Unified embedding for entropy-rate decomposition.** The identity $h = H(X_t) - I(X_t; X_{\text{past}})$ holds in expectation but is sensitive to finite-sample alignment. Computing H and I on different subsets of the data introduces a stochastic bias that does not vanish with N , because the digamma terms depend on the specific point set. A unified embedding that feeds both estimators from the same data matrix eliminates this source of error.

4.2 Remaining discrepancies

The Sample Entropy (SampEn) estimates show weak agreement (Spearman $\rho = 0.20$) between the two implementations. This reflects genuinely different algorithmic conventions in the template-matching step (inclusive vs. exclusive boundary, self-match handling) rather than a bug, and is not pursued further here.

4.3 Performance

The pure Python implementation runs approximately $12\times$ slower than the C/Cython version on this dataset (8.3s vs. 0.7s for 251 segments). This is acceptable for research and validation purposes but may require the optimized `core_optimized` module (using `scipy.spatial.cKDTree` and parallel workers) for production workloads.

5 Conclusion

We have validated a pure Python implementation of k-NN entropy and entropy-rate estimators against a reference C/ANN implementation, achieving numerical equivalence ($R^2 > 0.999$) on real-world physiological time-series data. The three implementation-level discrepancies identified—zero-distance handling, distance-metric mismatch, and embedding misalignment—are general pitfalls that apply to any reimplementations of k-NN information-theoretic estimators. The corrected Python codebase provides a readable, dependency-minimal reference implementation suitable for research, teaching, and cross-validation of compiled libraries.

All code and validation scripts are available at <https://github.com/martinfrasch/entropy> (branch `claude/refactor-python-efficiency-UGyKA`).

Acknowledgments

The systematic debugging and validation was performed with assistance from Claude Code (Anthropic), which identified the ANN library’s zero-distance exclusion mechanism as the primary root cause.

References

- L. F. Kozachenko and N. N. Leonenko. Sample estimate of the entropy of a random vector. *Problemy Peredachi Informatsii*, 23(2):9–16, 1987.
- A. Kraskov, H. Stögbauer, and P. Grassberger. Estimating mutual information. *Physical Review E*, 69(6):066138, 2004. doi: 10.1103/PhysRevE.69.066138.
- M. J. E. Mayer, N. B. Garnier, C. Becker, M. C. Antonelli, S. M. Lobmaier, and M. G. Frasch. Heart rate variability patterns reflect yoga intervention in chronically stressed pregnant women: A quasi-randomized controlled trial. *Bioengineering*, 12(11):1141, 2025. doi: 10.3390/bioengineering12111141.
- D. M. Mount and S. Arya. ANN: A library for approximate nearest neighbor searching. Technical report, University of Maryland, 2010. Version 1.1.2, <https://www.cs.umd.edu/~mount/ANN/>.